

Word Sense Disambiguation Combining Knowledge Graph and Text Hierarchical Structure

YUKUN CAO, Computer Science Department, Shanghai University of Electric Power, Shanghai, China
CHENGKUN JIN, Shanghai University of Electric Power, Shanghai, China
YIJIA TANG, Shanghai University of Electric Power, Shanghai, China
ZIYUE WEI, Shanghai University of Electric Power, Shanghai, China

Current supervised word sense disambiguation models have obtained high disambiguation results using annotated information of different word senses and pre-trained language models. However, the semantic data of the supervised word sense disambiguation models are in the form of short texts, and much of the corpus information is not rich enough to distinguish the semantics in different scenarios. This article proposes a bi-encoder word sense disambiguation method combining a knowledge graph and text hierarchy structure, by introducing structured knowledge from the knowledge graph to supplement more extended semantic information, using the hierarchy of contextual input text to describe the meaning of words and phrases, and constructing a BERT-based bi-encoder, introducing a graph attention network to reduce the noise information in the contextual input text, so as to improve the disambiguation accuracy of the target words in phrase form and ultimately improve the disambiguation effectiveness of the method. By comparing the method with the latest nine comparison algorithms in five test datasets, the disambiguation accuracy of the method mostly outperformed the comparison algorithms and achieved better results.

CCS Concepts: • **Computing methodologies** → **Language resources**;

Additional Key Words and Phrases: Word sense disambiguation, knowledge graph, BERT, graph attention network

ACM Reference Format:

Yukun Cao, Chengkun Jin, Yijia Tang, and Ziyue Wei. 2024. Word Sense Disambiguation Combining Knowledge Graph and Text Hierarchical Structure. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* 23, 12, Article 161 (November 2024), 16 pages. <https://doi.org/10.1145/3677524>

1 Introduction

Natural language is mostly ambiguous, but when words occur in a particular contextual context, the meaning of the target word is limited by the context, so it is possible to interpret the meaning of the target word according to the context in which it occurs. For example, what follows are contextual sentences containing the target word “cause”:

Authors’ Contact Information: Yukun Cao, Computer Science Department, Shanghai University of Electric Power, Shanghai, China; e-mail: marilyn_cao@163.com; Chengkun Jin, Shanghai University of Electric Power, Shanghai, China; e-mail: jinchengkun@163.com; Yijia Tang, Shanghai University of Electric Power, Shanghai, China; e-mail: tyj_sd@mail.shiep.edu.cn; Ziyue Wei, Shanghai University of Electric Power, Shanghai, China; e-mail: zfy362600403@163.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2375-4699/2024/11-ART161
<https://doi.org/10.1145/3677524>

- Those are causes for skepticism.
- Bad driving causes bad accidents.

The word “cause” has different meanings in these two sentences: “events that provide the generative force that is the origin of something” and “give rise to; cause to happen or occur, not always intentionally,” so the aim of disambiguation is to find the exact meaning of the ambiguous word in a particular context. Once the scope of the context has been determined, the choice of disambiguation method becomes the key to lexical disambiguation.

There are two main types of methods that work well for word sense disambiguation: supervised methods and knowledge-based methods. The supervised methods view word sense disambiguation as a classification problem that uses large batches of annotated semantic data as annotated data for word sense disambiguation to classify specific words to meanings defined by lexical dictionaries, and uses methods such as machine learning and neural networks to achieve word sense disambiguation. The knowledge-based methods use existing lexical resources such as knowledge bases and semantic networks as the disambiguation knowledge base and then determines the meaning of words based on the context.

However, supervised methods currently yield better disambiguation accuracy results than knowledge-based methods, but they are less flexible than knowledge-based methods for whole-word lexical disambiguation tasks. The supervised methods use short textual datasets, making much of the corpus less informative and making it difficult to distinguish semantics in different scenarios, which limits the speed of application and development of the supervised methods. At the same time, in word sense disambiguation tasks, the connection between words and phrases in the input text can make the model affect the disambiguation effect. When the target word of disambiguation is a word in a phrase, the connection between the phrase and the word can cause errors in the determination of word meaning, thus affecting the disambiguation effect.

To address the preceding problems, this article proposes a bi-encoder word sense disambiguation method based on a knowledge graph and text hierarchy structure. In an improved BERT-based bi-encoder model, the structured relational knowledge of the knowledge graph is embedded to combine the supervised and knowledge-based methods to enrich the dataset for word sense disambiguation and alleviates the problem of semantic differentiation in different scenarios. On this basis, this work designs a text hierarchy to represent contextual input text, distinguishes between words and phrases as target words, extracts association relations between words in phrases and phrases as a whole, clarifies the exact meaning of words in phrases in phrases, and enables the model to improve the disambiguation effect on phrases. Finally, as there are many adjacent words in the context of the word to be disambiguated, these adjacent words will affect the disambiguation effect, so a graph attention mechanism is introduced to aggregate these adjacent words to enhance the robustness of the model.

The main contributions of this article are as follows:

- A hierarchy of contextual input texts is constructed to extract the association between words in a phrase and the phrase as a whole, to explicitly describe the association between words and phrases, and to reduce the influence of individual words in a phrase on the interpretation of the phrase.
- Structured knowledge from knowledge graphs is introduced to enrich the semantic data of the model, and a mixture of supervised and knowledge-based methods is used to alleviate the problem of difficult data acquisition for supervised methods.

2 Related Work

At present, there are two mainstream methods of word sense disambiguation: knowledge-based and supervised methods.

Supervised word sense disambiguation methods often utilize large batches of annotated datasets with annotations, and use methods such as machine learning and neural networks to achieve word sense disambiguation. Context embedding is a new generation of semantic representation in the neural language models that solves the problem of semantic merging that hinders traditional word embedding. Loureiro and Jorge [15] use fully covered WordNet to create semantic level embedding without the help of explicit knowledge of semantic distribution or task-specific modeling. It is well proved that context embedding can have encouraging results in word sense disambiguation tasks. Some studies have shown that it is effective to incorporate gloss (sense definition) into the neural network for word sense disambiguation. Huang et al. [10] construct context-gloss pairs from the annotations of all possible semantics of the target word in WordNet to make better use of annotation information in a supervised neural network word sense disambiguation system, thus viewing word sense disambiguation as a sentence pair classification problem. The BEM model proposed by Blevins and Zettlemoyer [5] independently embeds the dictionary definition and annotation of the target word and its surrounding context and each semantics so that the encoder is jointly optimized in the same representation space, thus performing lexical disambiguation by finding the nearest semantics for each target word. In addition, Conia and Navigli [6] assigned multiple semantics to each target word and treated word sense disambiguation as a multi-label classification problem.

The knowledge-based methods mainly use existing vocabulary resources, such as WordNet [19], HowNet [8], and other knowledge graphs. This kind of method can use the graph structure of the semantic network to identify the most appropriate sense. Compared with supervised methods, such methods can use structured knowledge to obtain comprehensive semantic coverage and good performance. Scozzafava et al. [29] used the disambiguation word pairs contained in SyntagNet [18], a lexical semantic combination resource. The disambiguation accuracy of this method in a multi-lingual environment has gradually narrowed the gap between word sense disambiguation with the supervised method. The context representations derived from the neural language model have been proved to effectively represent the differences between different meanings of the same word. Still, these context representations are not easy to connect with the semantic network, so it is easy to ignore the information of the knowledge base itself. The SensEmBERT model proposed by Scarlini et al. [28] combines the expressive ability of language modeling with a large amount of knowledge in the semantic network to produce high-quality latent semantic representation in multiple languages. Additionally, since the vectors of this model are located in a vector space similar to the contextual word embedding, a simple nearest neighbor method can be used to make the disambiguation and its most likely semantic connection. However, the supervised method is difficult to obtain due to the manually annotated annotation data, whereas the knowledge-based method can make use of readily available data, so introducing knowledge graphs based on the supervised method and using this hybrid supervised and knowledge-based method can alleviate the difficulty of obtaining data for the supervised method.

In this article, we focus on how to introduce knowledge graphs into supervised methods. Inspired by recent research showing that knowledge graphs have yielded state-of-the-art results in many natural language processing tasks, such as entity alignment [35], text entailment [30], recommendation systems [34], and link prediction [11], we have successfully applied knowledge graphs to a supervised word sense disambiguation task. In our model, we also process words and phrases in manually annotated contextual input texts, making the dataset more suitable for the application of the word sense disambiguation model in this article.

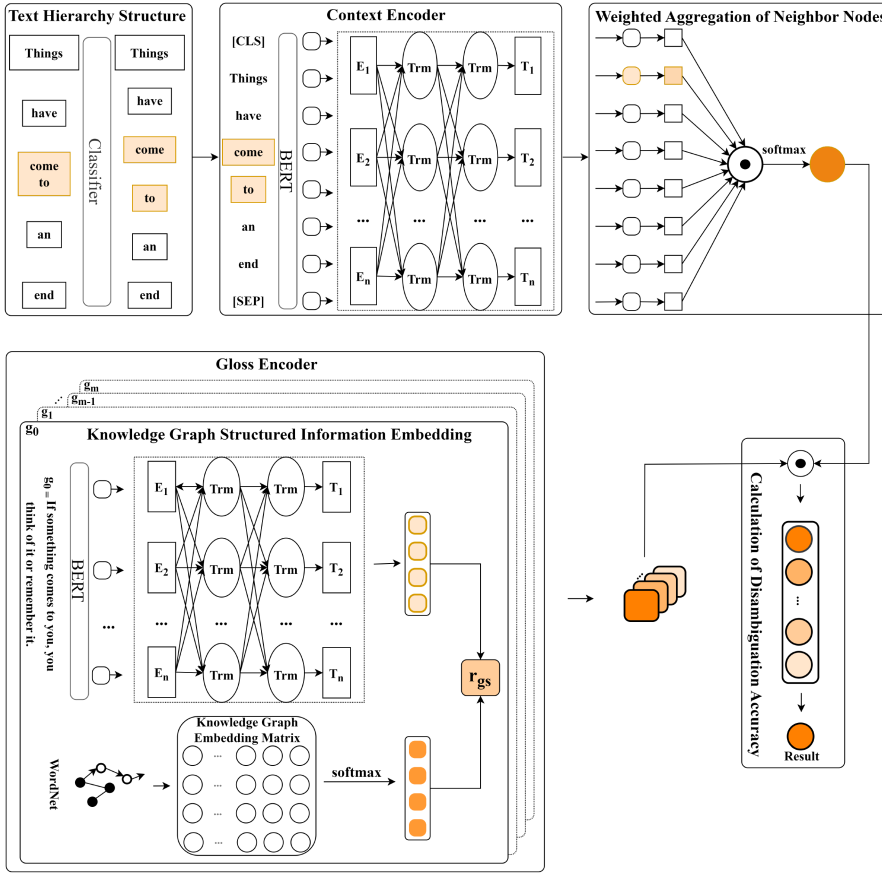


Fig. 1. An overall demonstration of our proposed model.

3 Model

3.1 Model Framework

The overall structure of a knowledge graph and text hierarchy based bi-encoder word sense disambiguation method proposed in this article is shown in Figure 1. It is divided into four main parts—knowledge graph structured information embedding, text hierarchy structure, BERT-based bi-encoder (containing context encoder and gloss encoder), and weighted aggregation of neighboring nodes:

Step 1—Knowledge graph structured information embedding: The knowledge graph is embedded in a vector representation of the target words by means of an adjacency matrix.

Step 2—Text hierarchy structure: Construct a hierarchy of phrases and their associated words from contextual input text.

Step 3—BERT-based bi-encoder: The BERT-based bi-encoder consists of two parts, which include context encoder and gloss encoder, where the contextual input text constructed in Step 2 is fed into the context encoder. The structured information and annotation data of the knowledge graph extracted in Step 1 are input into the gloss encoder. Finally, feature information is extracted from the context encoder and the gloss encoder, respectively.

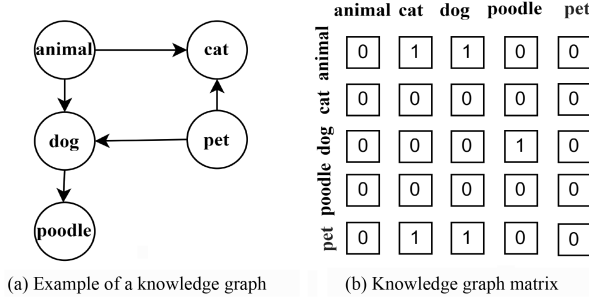


Fig. 2. Example diagram of knowledge graph embedding matrix.

Step 4—Weighted aggregation of neighboring nodes. The text features extracted from the context encoder are weighted and aggregated by the graph attention network for neighboring nodes, thus eliminating the influence of noisy neighboring nodes.

Step 5—Disambiguation accuracy calculation: The final disambiguation score is obtained by dot producing the weighted aggregated feature vector output in Step 4 with the feature vector of structured information from the knowledge graph output by the gloss encoder.

3.2 Knowledge Graph Structured Information Embedding

The supervised word sense disambiguation method is quite time-consuming and labor-intensive to obtain large batches of annotated data, so this article proposes to embed structured information from the knowledge graph into the annotated data and use the existing knowledge graph data to alleviate the difficulty of obtaining large batches of data.

The supervised word sense disambiguation approach treats word sense disambiguation as a token classification problem, similar to the lexical annotation task and the named entity recognition task in natural language processing. To implement word sense disambiguation requires generating a vector representation of the target word in a given context and using it to produce a probability distribution of all of its possible semantics. In this article, inspired by the paper presented by Bevilacqua and Navigli [4], the knowledge graph is embedded in the annotated data through the form of an adjacency matrix.

In this work, the knowledge graph embedding matrix A is built from the relationships between words and semantics in WordNet. Since there is a subordination relationship between words, for example, the word “animal” in Figure 2 is the hypernym of the words “dog” and “cat,” and the word “poodle” is the hyponym of the word “dog,” a word has its hypernym and hyponym, so when constructing the knowledge graph embedding matrix A , the superior and inferior relations between words are considered. The knowledge graph embedding matrix is constructed by experimenting with only the superlative, only the subordinate, and both hypernym and hyponym. An example of the approximate construction process is shown in Figure 2, where Figure 2(a) presents an example diagram, and Figure 2(b) presents a knowledge graph embedding matrix constructed from the example diagram.

To embed the knowledge graph, the constructed knowledge graph embedding matrix A and the annotated vector representation matrix of the words to be disambiguated and its weight matrix and deviation vector are subjected to a dot product operation to obtain the final knowledge graph vector matrix Q in this article. The process is represented by Equation (1):

$$Q = (HO + b_q) A^T + (HO + b_q), \quad (1)$$

where Q denotes the vector matrix of the knowledge graph obtained. Since each word to be disambiguated has multiple annotations, each annotation vector to be disambiguated has a vector representation. H is denoted as the matrix of vector representations of all annotations of the word to be disambiguated, O denotes the weight matrix of the word to be disambiguated in the knowledge graph, and b_q denotes the bias vector.

To achieve word sense disambiguation requires generating a vector representation of the word to be disambiguated in a given context and using it to generate a probability distribution of all of its possible semantics. To be able to use the information in the knowledge graph $G = \langle V, E, w \rangle$, where V represents the set of all vertices in the knowledge graph, E represents all edges, and w represents the weight of the edge between two vertices. A vector score is calculated for each synonym of the word, and the final vector q_s is obtained by adding the vector score of the word to be disambiguated to the weight of the edges between the word and all of its synonyms. To express this process in terms of Equation (2):

$$q_s = z_s + \sum_{s' \in V | \langle s', s \rangle \in E} w(\langle s', s \rangle) \cdot Z_{s'}, \quad (2)$$

where $\langle s', s \rangle$ denotes the edge between the synonyms of each word and q_s denotes the weighted combination of all output lexical vector scores. $w(\langle s', s \rangle)$ denotes the weight of the edge between the word s to be disambiguated and all of its synonyms s' . Z_s denotes the vector representation matrix of the word s to be disambiguated plus the result of annotating the embedding vector on the dot product of the deviation vector columns. We express this process in Equation (3):

$$z_s = h^T g_{(s)} + b^T g_{(s)}, \quad (3)$$

where $g_{(s)}$ denotes the annotated embedding vector.

Finally, the vector of the knowledge graph embedding matrix Q normalized by the softmax function is spliced on its own vector score q_s to obtain the final feature vector score r_{q_s} in the gloss encoder.

3.3 Text Hierarchy Structure

When selecting a target word from input text, the target word may be a word or a phrase, but the phrase often contains multiple words, which often affect the word meaning judgment of the phrase. Therefore, this article detects the degree of interaction between words and phrases through recursion, then divides larger text partitions into smaller text partitions based on this degree of impact to build a hierarchical structure.

For the word sense disambiguation task, a sentence c containing a set of words s to be disambiguated is taken as input. Define c as follows: $c = c_0, c_1, \dots, c_i, \dots, c_n$, where c_i is the target word to be disambiguated at position i in the sentence. In addition, the partition $P = \{c_{(0, m_1]}, c_{(m_1, m_2]}, \dots, c_{(m_{i-1}, n]}\}$ is defined to denote a sentence partition with sentence length n , where $c_{(m_i, m_{i+1}]} = (c_{(m_i)+1}, \dots, c_{m_{i+1}})$. c_{m_i} denotes the words at either end of each partition of the sentence. The basic process of partitioning is to divide the given larger partition $c_{(m_i, m_{i+1}]}$ into two smaller partitions $c_{(m_i, j]}$ and $c_{(j, m_{i+1}]}$, where j denotes the cut-off point ($m_i < j < m_{i+1}$). Figure 3 illustrates the process of dividing the sentence “The exam results “come out” in August.”

Algorithm 1 describes the entire process of dividing the contextual sentence c into different phrasal partitions. Starting with the whole sentence, the algorithm first divides c into two segments. In the next iteration, it selects one of the two segments and further splits it into smaller partitions. The final step of each iteration is to evaluate the correlation between the new partitions and update the relevance set R , as shown in line 6 of the algorithm.

The final output of Algorithm 1 consists of the relevance set R and the hierarchy set H . The relevance set R contains all text partitions generated at each division and the number of relevance

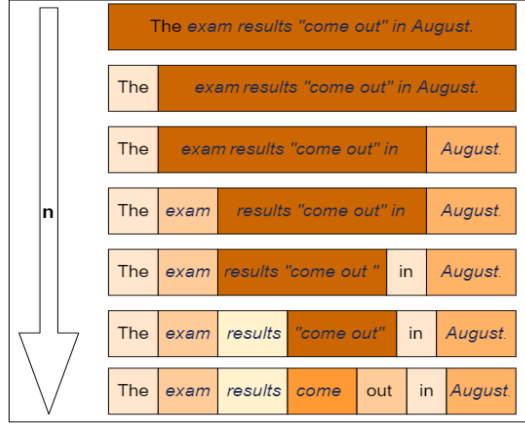


Fig. 3. Text hierarchy structure layering process.

ALGORITHM 1: Constructing text hierarchical structure.**Input:** Contextual text c , sentence length n **Output:** The relevance set R , the hierarchy set h

- 1: Initialize the partition length $P_0(0, n]$, the relevance set $R_0 = \emptyset$, the hierarchy set $H = [P_0]$
- 2: **for** $t = 1, \dots, n - 1$ **do**
- 3: Update the partition length P_t , P'_t is obtained by dividing $c_{(m_i, m_{i+1}]}$ by P_{t-1} , and then P'_t is merged with $\{C_{(m_i, j]}, C_{(j, m_{i+1}]}\}$ to obtain P_t .
- 4: $H.add(P_t)$
- 5: Update the relevance set R , R'_t is obtained by dividing $\{(c_{(m_i, j]}, \psi(c_{(m_i, j]}))\}$ by R_{t-1} , and then R'_t is merged with $\{(c_{(j, m_{i+1}]}, \psi(c_{(j, m_{i+1}]})\}$ to obtain R .
- 6: **end for**
- 7: **return** R, H

links between words and phrases between partitions, and the hierarchy set H contains all the partitions of the contextual sentence c at each division.

3.4 BERT-Based Bi-encoder

The bi-encoder structure encodes the target word context and its gloss separately, with each encoder initialized using a pre-trained language model BERT. Thus, the encoder inputs require BERT-specific start symbols [CLS] and end symbols [SEP], and the part of the bi-encoder structure that deals with context is called the *context encoder* and the part that deals with semantic gloss is called the *gloss encoder*.

Contextual input text is processed by a hierarchical structure and fed into the context encoder. When the disambiguation target word is a single word, the [CLS] and [SEP] symbols are added to the input text and then fed into the pre-trained language model BERT encoder. When the disambiguation target word is a phrase, the vector average of each word contained in the phrase is used to represent the phrase, as shown in Equation (4):

$$r_{ws} = \frac{1}{q-j} \sum_{l=j}^q (T_c(c)[l]), \quad (4)$$

where j and q denote the first and last words of the partition span P of this phrase, T_c denotes the context encoder, and $[l]$ denotes putting each word into the context encoder in turn. r_{ws} denotes the feature vector obtained from the input text after the context encoder.

In the gloss encoder, the semantic gloss $g_s = g_0, g_1, \dots, g_m$ of the word s to be disambiguated is used as input. The gloss encoder generates a feature vector $g_{(s)}$ representing the semantic s by means of Equation (5):

$$g_{(s)} = T_g(g_s)[0], \quad (5)$$

where T_g denotes the gloss encoder. The first vector of the output of the gloss encoder (corresponding to the input [CLS] token) is taken as the global vector.

The gloss embedding vector score in the gloss encoder and the normalized result of the knowledge graph embedding matrix are then stitched together to obtain the final output feature vector r_{gs} .

3.5 Neighbor Node Weighted Aggregation

As there are many neighboring words around the context of the word to be disambiguated, the presence of noisy information in these neighboring words will affect the disambiguation accuracy of the model for the word to be disambiguated. This makes the output of the context encoder model less robust, so this work uses a graph attention network to weight the neighboring nodes for aggregation to eliminate the effect of noisy neighbors on accuracy.

The graph attention network used in this article is calculated in two main steps. The first step is to calculate the attention coefficient, and for the word s to be disambiguated, the similarity coefficient between its neighbors and itself is calculated one by one, and the similarity coefficient is shown in Equation (6):

$$\alpha_{sj} = \text{LeakyReLU} \left(h \left([Wr_{ws} \| Wr_{wj}] \right) \right), j \in N_i, \quad (6)$$

where r_{ws} denotes the feature vector obtained from the output of the context encoder, and N_s denotes the set of all neighboring nodes of the word to be disambiguated. Wr_{ws} denotes the value obtained by linear transformation of r_{ws} and h denotes a learnable weight vector. $[Wr_{ws} \| Wr_{wj}]$ denotes the splicing operation after the feature transformation of r_{ws} and r_{wj} ; LeakyReLU is the activation function.

Once the similarity coefficient α_{sj} is obtained, the attention coefficient β_{sj} is then calculated by the normalization operation, and Equation (7) represents the calculation process:

$$\beta_{sj} = \frac{\exp(\text{LeakyReLU}(e_{sj}))}{\sum_{k \in N_s} \exp(\text{LeakyReLU}(e_{sk}))}. \quad (7)$$

After obtaining the attention coefficients β_{sj} in the first step, a new feature vector is obtained by weighting and summing the attention coefficients β_{sj} in the second step. The calculation process is shown in Equation (8):

$$r_{ws}' = \sigma \left(\sum_{j \in N_s} \beta_{sj} Wr_{wj} \right), \quad (8)$$

where r_{ws}' denotes the new feature representation output by the graph attention network for each word s after fusing neighborhood information and $\sigma(\cdot)$ denotes the activation function.

Finally, the new feature representation r_{ws}' obtained from the feature weighted summation is subjected to a feature enhancement operation to make the result more robust. The final obtained

Table 1. Datasets Statistics

Dataset	Noun	Verb	Adj	Adv	Average sentence length
SE2	1,066	517	445	254	6
SE3	900	588	350	12	3
SE07	159	296	0	0	5
SE13	1,644	0	0	0	7
SE15	531	251	160	80	5

$r_{w_s}'(K)$ is the feature vector representation of the final output of the encoder. The calculation process is shown in Equation (9):

$$r_{w_s}'(K) = \parallel_{k=1}^K \sigma \left(\sum_{j \in N_s} \beta_{sj}^k W^k r_{wj} \right), \quad (9)$$

where K denotes the number of heads of the multi-headed attention mechanism.

3.6 Disambiguation Accuracy Calculation

The feature vector of the final output of the encoder part is denoted as $r_{w_s}'(K)$, and the final output feature vector of the gloss encoder is denoted as r_{gs} . The final disambiguation accuracy in this work is obtained by dot producing $r_{w_s}'(K)$ and r_{gs} as shown in Equation (10):

$$\emptyset(w, s_i) = r_{w_s}'(K) \cdot r_{si}. \quad (10)$$

In this article, the model is trained using the cross-entropy loss of the candidate semantic scores of the word s to be disambiguated. The loss function used is shown in Equation (11):

$$\mathcal{L}(s, g_i) = -\phi(s, g_i) + \log \sum_{j=0}^{|g_s|} \exp(\phi(s, g_j)), \quad (11)$$

where (s, g_i) denotes the word and its gloss.

4 Experiments

4.1 Datasets

In this work, the model was trained on the SemCor [20] dataset. The SE07 (SemEval-2007) [24] dataset was then used as the development set for the model. In particular, the SemCor dataset contains 226,036 annotated examples covering 33,362 individual word senses. Additionally, SE2 (Senseval-2) [23], SE3 (Senseval-3) [31], SE13 (SemEval-2013) [22], SE15 (SemEval-2015) [21], and a tandem dataset (ALL) of all preceding datasets were used as the test set for the model, and the specific details of these datasets are shown in Table 1, where the average sentence length represents the average of the sentence lengths in the dataset. All semantic data used in the model were obtained from WordNet 3.0, and the knowledge graph information used in the model was also obtained from WordNet 3.0 [26].

4.2 Experimental Setup

The experimental environment for all experiments in this article was a Tesla T4 graphics card. The BERT pre-trained model dimension was 768. The experimental optimizer was chosen from Adam [12], the learning rate during training was 0.00001, the batch size used for training the context encoder was 4, the batch size used for training the gloss encoder was 256, the maximum number of iterations was 20, and the number of layers of the graph attention network was 2.

4.3 Baselines

In this article, the model is compared with the six most recent methods on six datasets and the F1 value is used as an evaluation metric:

- (1) The CAN [16] model introduces a common attention mechanism that generates common dependency representations for both gloss-context and context-gloss.
- (2) The HCAN [16] model extends the CAN model into a hierarchy that allows the model to capture not only local word-level features but also global sentence-level features.
- (3) GASext (Concatenation) [17] uses joint encoding of relationships between context and gloss of target words in a modified memory network framework.
- (4) The GASext (Linear) [17] model updates the memory variables during joint coding by means of a linear transformation.
- (5) The EWISE [13] model is a supervised model that performs word sense disambiguation by predicting a continuous semantic embedding space rather than a discrete label space.
- (6) The LMMS [15] model uses contextual embeddings, annotations, and semantic networks to achieve full coverage of all WordNet semantics.
- (7) The GLU [9] model incorporates BERT, which effectively captures the context and linear projection in the hidden vector, as well as the selection process used to filter the values.
- (8) The GlossBERT [10] model treats the word sense disambiguation region as a sentence pair classification problem, constructs contextual annotation pairs, and investigates the full use of gloss data through various neural network models.
- (9) The BEM [5] model independently encodes the target word context and the gloss information of the target word, and jointly encodes the final features to obtain the most appropriate word meaning for the target word.
- (10) The ESC [2] model improves model performance and generalization by redefining WSD as a span extraction problem, capable of observing both input sentences and sense definition candidates.
- (11) The SCSMM [1] model combines semantic similarity, heuristic knowledge, and document context to disambiguate using local meaning-based context between consecutive terms, human knowledge about the terms, and the main idea of the document, respectively.
- (12) The B-reweighting [32] strategy is applied at the semantic level. For each word, the semantic weight is proportional to the inverse of the training instance.
- (13) The B-resampling [32] method is applied at the word level. Each word is first sampled with the same probability. The training cases for the selected words are then sampled randomly. The standard cross-entropy loss is used in this method.
- (14) The LDAM [32] method adapts training at the semantic level. The goal of LDAM is to solve the class imbalance problem by exploiting the marginal loss perceived by the label distribution.
- (15) Z-reweighting [32] proposes a Z-reweighting method at the word level to adjust training on unbalanced datasets based on the statistical relationship between word frequency classes and the distribution of the number of word senses.
- (16) The EWISER [4] model is a neural supervised architecture that can mine this rich knowledge by embedding information from LKB graphs in the neural architecture, and utilize pre-trained syntactic set embedding to enable networks to predict syntactic sets that are not in the training set.
- (17) The ConSeC [3] model takes advantage of the recent reframing of the task as a text extraction problem by applying it to the formulation of the word sense disambiguation task and

Table 2. Results of All Methods Compared on the F1 Score for the Baseline Model Results

Model		SE07	SE3	SE13	SE15	ALL
Supervised	CAN	—	70.2*	69.1*	72.2*	70.9*
	HCAN	—	70.3*	68.5*	72.8*	71.1*
	GASeXt (Concatenation)	—	70.5*	67.2*	72.6*	70.6*
	GASeXt (Linear)	—	70.1*	67.1*	72.1*	70.4*
	EWISSE	67.1	70.5	68.8	73.7	71.3
	LMMS	67.9	74.8	74.3	76.1	74.8
	GLU	67.8	72.9	70.3	75.4	73.2
	GlossBERT	71.6	75.9	76.8	78.9	76.1
	BEM	73.0	77.5	78.5	80.3	78.1
	ESC	73.9	77.8	79.2	80.8	78.2
	ConSeC	<u>74.0</u>	78.2	<u>79.4</u>	81.0	<u>78.3</u>
Knowledge Base	SCSMM	64.78*	72.67*	63.44*	68.38*	69.67*
	B-resampling	60.4±0.6*	68.8±1.1*	72.8±1.2*	74.7±0.9*	70.9±0.1*
	B-reweighting	71.3±0.3*	75.5±0.5*	75.6±1.3*	80.4±1.3*	77.1±0.3*
	LDAA	71.3±0.3*	75.4±0.6*	76.6±0.6*	80.3±0.2*	77.1±0.1*
	Z-reweighting	71.9±0.5*	76.5±0.2*	78.9±0.5*	82.5±0.9*	78.6±0.2*
	EWISER	71.0*	78.4*	78.9*	79.3*	78.3*
Ours		74.3	<u>77.0</u>	79.6	<u>79.9</u>	78.5

An asterisk (*) indicates retrieval from the original paper. Results without a symbol indicate that the paper was run through the paper in a local experimental environment. A dash (—) indicates that the result was not reported in the original paper. The results of the work's model are shown in the last row. The best results on each dataset are shown in bold, and the second best results are underlined.

introducing a feedback loop strategy so that target word disambiguation depends not only on its context but also on the explicit meaning assigned to nearby words.

Table 2 shows the disambiguation accuracy scores of the different methods on the five datasets, and it can be seen that the disambiguation accuracy of the model in this work is mostly better than the latest models. Although the disambiguation accuracy of this model was not optimal in two datasets (SE15 and SE3) due to the large proportion of adjectives and adverbs in these two datasets compared to the other datasets and the lowest average sentence length in these two datasets, the disambiguation accuracy of this model was optimal in all other datasets. The main reason this model outperforms other models is that it not only uses the pre-trained language model BERT but also uses a dual encoder to encode contextual and annotated information separately, and has a hierarchy of explicit representations between words and phrases used for contextual input text information, which improves the model's ability to disambiguate phrases. In addition, the model incorporates structured information from the knowledge graph and introduces a graph attention mechanism for noisy data in the input text, making the disambiguation accuracy of this model better than other models.

Table 3 shows the results of the experiments comparing the model in this work with the comparison model on a dataset consisting of four types of words (noun, verb, adjective, and adverb) in the ALL dataset. It can be seen that the disambiguation accuracy scores of the models in this article are better than those of the other models.

Figure 4 visually depicts the lexical distribution in each dataset and shows the disambiguation accuracy values obtained by the proposed method. It can be seen that the percentage of verbs in the SE07 dataset is the largest, but the value of disambiguation accuracy is the smallest. The percentage of verbs in the SE2, SE3, and SE15 datasets is similar and the value of disambiguation accuracy is not very different.

Table 3. Results of All Methods Compared on the Disambiguation Accuracy Score

Model	Noun	Verb	Adjective	Adverb
CAN	73.5	56.5	76.6	80.3
HCAN	72.7	58.2	77.4	84.1
GAStxt (Concatenation)	72.2	57.7	76.6	84.7
GAStxt (Linear)	71.9	58.1	76.4	84.7
EWISSE	74.0	60.2	78.0	82.1
GlossBERT	79.8	67.1	79.6	87.4
BEM	81.4	68.5	83.0	87.9
Ours	81.9	69.1	83.5	88.3

Table 4. Results of This Article’s Model Using Different Pre-Trained Language Models

	SE07	SE2	SE3	SE13	SE15	ALL
BERT [7]	74.3	79.1	77.0	79.6	79.9	78.5
RoBERTa [14]	74.1	78.9	78.5	78.2	79.8	78.3
DistilBERT [27]	71.6	79.3	76.5	79.0	81.2	78.3
GPT-2 [25]	72.5	77.4	77.9	76.8	78.6	77.6

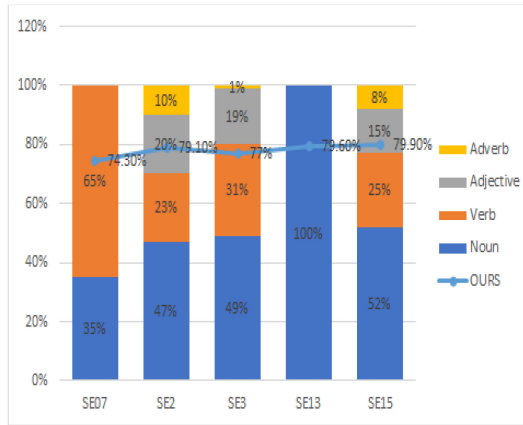


Fig. 4. Part of speech distribution in each dataset and example diagram of model results in this article.

5 Analysis

5.1 Pre-Trained Language Model Selection

This work uses the pre-trained language model BERT and therefore considers the impact of the choice of different pre-trained language models on the models in this article. Table 4 shows the results of using different pre-trained language models for the models in this work. All models used are the base version. From Table 4, it can be seen that applying the BERT model to the models in this article gives the best results for the models. The reason for the superiority of the BERT model over the other models is that the corpus dataset used in the training of RoBERTa and others is 512 in length, whereas the datasets used for the word sense disambiguation task in this work are no more than 10 in length, so the BERT model works better than the other pre-trained models.

Table 5. Results of the Ablation Experiment

	SE07	SE2	SE3	SE13	SE15	ALL
Full	74.3	79.1	77.0	79.6	79.9	78.5
W/O Context Encoder	70.5	75.3	72.5	74.6	73.8	75.3
W/O Gloss Encoder	66.1	70.4	69.2	69.5	67.4	68.3
W/O Context Hierarchical structure	73.8	78.9	76.5	79.1	79.2	77.0
W/O Knowledge Graph Embedding	73.5	78.2	76.3	78.5	78.8	77.9
W/O GAT [33]	73.4	78.1	76.4	77.9	78.4	77.6

5.2 Ablation Study

To understand the impact of each module of the model on the disambiguation accuracy, the degree of impact of each module on the model was analyzed by disambiguating each part of the model. Table 5 shows the impact of each module on the disambiguation accuracy of the model, where Full indicates that all modules are present and W/O indicates that the module is not present. It can be seen that the contextual and annotation encoders have the greatest impact on the disambiguation accuracy of the model and are the main components of the model. The disambiguation accuracy of the hierarchical structure module on the model decreases significantly on the ALL dataset, indicating that the impact of the hierarchical structure module on the disambiguation accuracy of the model is influenced by the number of phrases in the dataset, which also has a greater impact on the disambiguation accuracy of the model. The knowledge graph structured information embedding module and the graph attention mechanism module had some impact on all datasets.

5.3 Case Study

Table 6 lists all semantics of the word “root” with and without the knowledge graph. It can be seen that the addition of the knowledge graph enriches the semantics of the word “root,” which allows for better disambiguation of the semantics of the contextual statements. For example, when the phrase “My dentist pre-scribed me a root canal procedure” is encountered, the word “root” has no semantic meaning without the addition of the knowledge map. However, with the addition of the information from the knowledge graph, the word “root” is given the new semantic meaning, “the part of a tooth that is embedded in the jaw and serves as support,” which is a more accurate translation of the word “root” at this point in time.

Figure 5(a) shows the visualization of the two semantics of the word “use” in the SemCor dataset in this model and the baseline model. As can be seen, although the baseline model can also distinguish between different semantics of words, it does not aggregate all of the same semantics very well. However, the model in this work not only distinguishes the different semantics better but also aggregates the same semantics to a higher degree.

Figure 5(b) shows a visualization of the two semantics of the phrase “come to” in the SemCor dataset for both this model and the baseline model. As can be seen, the number of points in Figure 6 is smaller compared to the results for the word “use” in Figure 5. While the visualizations in the baseline model can also distinguish between the different semantics of the phrases, the model in this article distinguishes more strongly between the different semantics of the phrases and aggregates the same semantics to a greater extent. This can be explained by the fact that the hierarchical explanatory structure module in the disambiguation experiments did not have a sufficient impact on the model because the number of phrases in the dataset was much smaller than the number of words, and therefore the disambiguation accuracy of this module did not have as significant an impact on the model as the other modules.

Table 6. List of All Semantics of the Word “Root” with and without Knowledge Graphs

Without Knowledge Graph	With Knowledge Graph
the place where something begins, where it springs into being	the place where something begins, where it springs into being
(botany) the usually underground organ that lacks buds or leaves or nodes; absorbs water and mineral salts; usually it anchors the plant to the ground	(botany) the usually underground organ that lacks buds or leaves or nodes; absorbs water and mineral salts; usually it anchors the plant to the ground
(linguistics) the form of a word after all affixes are removed	(linguistics) the form of a word after all affixes are removed
the set of values that give a true statement when substituted into an equation	the set of values that give a true statement when substituted into an equation
a number that, when multiplied by itself some number of times, equals a given number	a number that, when multiplied by itself some number of times, equals a given number
become settled or established and stable in one’s residence or life style	become settled or established and stable in one’s residence or life style
take root and begin to grow	take root and begin to grow
	the part of a tooth that is embedded in the jaw and serves as support
	dig with the snout
	come into existence, originate
	cause to take roots

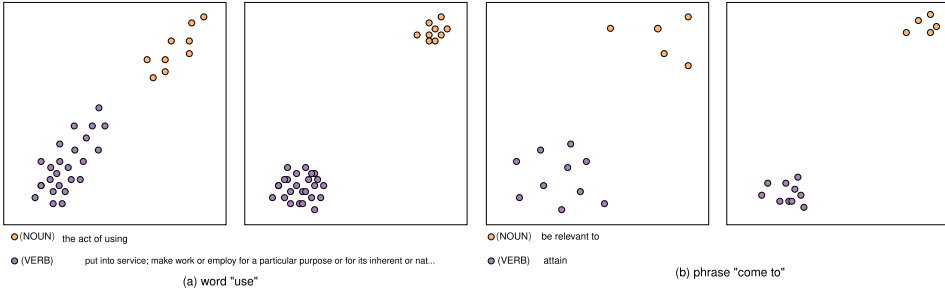


Fig. 5. Visualization of the results of the two semantics of the word “use” in this article’s model and in the baseline model.



Fig. 6. The influence of context on the meaning of words when the word “bank” has different meanings in different sentences.

Figure 6 shows how the adjacency of the word “bank” in a contextual sentence affects the interpretation of the word to varying degrees. The word “bank” in Figure 6(a) is used in this context as “a financial institution that accepts deposits and channels the money into lending activities,” where the word “check” has the greatest influence on the interpretation of the word. When the

graph attention network module is added, the accuracy of determining the meaning of the target word is deepened. The word “bank” in Figure 6(b) is interpreted as “sloping land (especially the slope beside a body of water),” where the word “canoe” has the greatest influence on the interpretation of the word. With the addition of the graph attention network module, the accuracy of determining the meaning of the target word “bank” is increased accordingly.

6 Conclusion

This article proposed a bi-encoder word sense disambiguation method based on knowledge graph and text hierarchy, which alleviates the problem of difficult data acquisition with supervised methods. The disambiguation capability of the model is enhanced by introducing structured information from the knowledge graph to enhance the semantic annotation information. To reduce the impact of noisy data on the model and to enhance the robustness of the model, a graph attention mechanism is introduced into the model. The model also designed a hierarchy for contextual input text, which enhanced the disambiguation effect of the model on phrases by considering the correlation between words and phrases. The disambiguation accuracy of the model in this work was improved on all five test sets. However, this article only explored a limited number of ways to improve the performance of word sense disambiguation, and there is more room for improvement in word sense disambiguation. For future work, more methods can be applied to word sense disambiguation to further improve the performance of the word sense disambiguation model.

References

- [1] M. Almousa, R. Benlamri, and R. Khoury. 2022. A novel word sense disambiguation approach using WordNet knowledge graph. *Computer Speech & Language* 74 (2022), 101337.
- [2] Edoardo Barba, Tommaso Pasini, and Roberto Navigli. 2021. ESC: Redesigning WSD with extractive sense comprehension. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4661–4672. <https://doi.org/10.18653/v1/2021.naacl-main.371>
- [3] Edoardo Barba, Luigi Procopio, and Roberto Navigli. 2021. ConSeC: Word sense disambiguation as continuous sense comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 1492–1503. <https://doi.org/10.18653/v1/2021.emnlp-main.112>
- [4] M. Bevilacqua and R. Navigli. 2020. Breaking through the 80% glass ceiling: Raising the state of the art in word sense disambiguation by incorporating knowledge graph information. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2854–2864.
- [5] T. Blevins and L. Zettlemoyer. 2020. Moving down the long tail of word sense disambiguation with gloss-informed biencoders. *arXiv:2005.02590* (2020).
- [6] Simone Conia and Roberto Navigli. 2021. Framing word sense disambiguation as a multi-label problem for model-agnostic knowledge integration. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL '21)*. 3269–3275.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [8] Z. Dong and Q. Dong. 2003. HowNet—A hybrid language and knowledge resource. In *Proceedings of the 2003 International Conference on Natural Language Processing and Knowledge Engineering*.
- [9] Christian Hadiwinoto, Hwee Tou Ng, and Wee Chung Gan. 2019. Improved word sense disambiguation using pre-trained contextualized word representations. *arXiv preprint arXiv:1910.00194* (2019).
- [10] L. Huang, C. Sun, X. Qiu, and X. Huang. 2019. GlossBERT: BERT for word sense disambiguation with gloss knowledge. *arXiv:1908.07245* (2019).
- [11] Seyed Mehran Kazemi and David Poole. 2018. Simple embedding for link prediction in knowledge graphs. *arXiv preprint arXiv:1802.04868* (2018).
- [12] D. Kingma and J. Ba. 2014. Adam: A method for stochastic optimization. *arXiv:1412.6980* (2014).
- [13] Sawan Kumar, Sharmistha Jat, Karan Saxena, and Partha Talukdar. 2019. Zero-shot word sense disambiguation using sense definition embeddings. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 5670–5681.

- [14] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [15] D. Loureiro and A. Jorge. 2019. Language modelling makes sense: Propagating representations through WordNet for full-coverage word sense disambiguation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.
- [16] Fuli Luo, Tianyu Liu, Zexue He, Qiaolin Xia, and Baobao Chang. 2018. Leveraging gloss knowledge in neural word sense disambiguation by hierarchical co-attention. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.
- [17] F. Luo, T. Liu, Q. Xia, B. Chang, and Z. Sui. 2018. Incorporating glosses into neural word sense disambiguation. In *Proceedings of the Meeting of the Association for Computational Linguistics*.
- [18] Marco Maru, Federico Scozzafava, Federico Martelli, and Roberto Navigli. 2019. SyntagNet: Challenging supervised word sense disambiguation with lexical-semantic combinations. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*.
- [19] George A. Miller. 1995. WordNet: A lexical database for English. *Communications of the ACM* 38, 11 (1995), 39–41.
- [20] George A. Miller, Claudia Leacock, Randee Teng, and Ross T. Bunker. 1993. A semantic concordance. In *Proceedings of the Workshop on Human Language Technology*.
- [21] Andrea Moro and Roberto Navigli. 2015. SemEval-2015 task 13: Multilingual all-words sense disambiguation and entity linking. In *Proceedings of the International Workshop on Semantic Evaluation*.
- [22] Roberto Navigli, David Jurgens, and Daniele Vannella. 2013. SemEval-2013 task 12: Multilingual word sense disambiguation. In *Proceedings of the 2nd Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the 7th International Workshop on Semantic Evaluation (SemEval'13)*. 222–231.
- [23] Martha Palmer, Christiane Fellbaum, Scott Cotton, Lauren Delfs, and Hoa Trang Dang. 2001. English tasks: All-words and verb lexical sample. In *Proceedings of the 2nd International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL'01)*.
- [24] Sameer Pradhan, Edward Loper, Dmitriy Dligach, and Martha Palmer. 2007. SemEval-2007 task-17: English lexical sample, SRL and all words. In *Proceedings of the 4th International Workshop on Semantic Evaluations (SemEval'07)*.
- [25] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
- [26] Alessandro Raganato, Jose Camacho-Collados, and Roberto Navigli. 2017. Word sense disambiguation: A unified evaluation framework and empirical comparison. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. 99–110.
- [27] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).
- [28] Bianca Scarlini, Tommaso Pasini, and Roberto Navigli. 2020. SensEmBERT: Context-enhanced sense embeddings for multilingual word sense disambiguation. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 5 (2020), 8758–8765.
- [29] Federico Scozzafava, Marco Maru, Fabrizio Brignone, Giovanni Torrisi, and Roberto Navigli. 2020. Personalized PageRank with syntagmatic information for multilingual word sense disambiguation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. 37–46.
- [30] V. S. Silva, A. Freitas, and S. Handschuh. 2019. Exploring knowledge graphs in an interpretable composite approach for text entailment. *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (2019), 7023–7030.
- [31] B. Snyder and M. Palmer. 2004. The English all-words task. In *Proceedings of the 3rd International Workshop on the Evaluation of Systems for the Semantic Analysis of Text (SENSEVAL'04)*.
- [32] Ying Su, Hongming Zhang, Yangqiu Song, and Tong Zhang. 2022. Rare and zero-shot word sense disambiguation using Z-Reweight. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4713–4723. <https://doi.org/10.18653/v1/2022.acl-long.323>
- [33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [34] Xiang Wang, Dingxian Wang, Canran Xu, Xiangnan He, and Tat Seng Chua. 2019. Explainable reasoning over knowledge graphs for recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (2019), 5329–5336.
- [35] Q. Zhang, Z. Sun, W. Hu, M. Chen, and Y. Qu. 2019. Multi-view knowledge graph embedding for entity alignment. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI'19)*.

Received 17 January 2022; revised 13 April 2023; accepted 25 May 2024